

THÔNG TIN CHUNG CỦA NHÓM

- Link YouTube video của báo cáo (tối đa 5 phút):

<https://youtu.be/vysUCWwLY-M>

- Link slides (dạng .pdf đặt trên Github của nhóm):

<https://github.com/longvuzz/CS2205.JAN2026/blob/main/V%C5%A9%20Nguy%E1%BB%85n%20Long%20-%20CS2205.JAN2026.DeCuong.FinalReport.Template.Slide.pdf>

- Họ và Tên: Nguyễn Long Vũ

- MSSV: 250202060



- Lớp: CS2205.JAN2026

- Tự đánh giá (điểm tổng kết môn): 9/10

- Số buổi vắng: 1

- Số câu hỏi QT cá nhân: 5

- Số câu hỏi QT của cả nhóm: 5

- Link Github:

<https://github.com/longvuzz/CS2205.JAN2026>

ĐỀ CƯƠNG NGHIÊN CỨU

TÊN ĐỀ TÀI

PHÁT HIỆN BẤT THƯỜNG HỆ THỐNG ĐÁM MÂY DỰA TRÊN PHÂN TÍCH
NGŨ NGHĨA NHẬT KÝ SỬ DỤNG LLM MÃ NGUỒN MỞ CHẠY CỤC BỘ

TÊN ĐỀ TÀI TIẾNG ANH

CLOUD SYSTEM ANOMALY DETECTION BASED ON SEMANTIC LOG
PARSING USING LOCAL OPEN-SOURCE LLMS

TÓM TẮT

Nhật ký hệ thống (system logs) là nguồn dữ liệu nền tảng phản ánh trạng thái hoạt động của các hệ thống cloud hiện đại. Các tác vụ AIOps quan trọng như phát hiện bất thường (anomaly detection) đều phụ thuộc vào chất lượng của bước tiền xử lý log log parsing. Tuy nhiên, các bộ phân tích log truyền thống chỉ trích xuất template cú pháp và thay thế tham số động bằng ký tự đại diện "<*>", bỏ qua toàn bộ ngữ nghĩa chứa đựng trong các tham số đó. Hệ quả là các mô hình phát hiện bất thường hạ nguồn mất đi một lượng lớn thông tin ngữ cảnh có giá trị. Mặt khác, các bộ phân tích log dựa trên LLM thế hệ mới như LILAC, LUNAR tuy đạt độ chính xác cao nhưng phụ thuộc vào API thương mại trực tuyến ChatGPT, gây rủi ro nghiêm trọng về bảo mật dữ liệu doanh nghiệp và chịu ảnh hưởng bởi độ trễ mạng.

Nghiên cứu này đề xuất một pipeline phát hiện bất thường end-to-end kết hợp hai thành phần:

- (1) tái hiện kiến trúc SemanticLog bộ phân tích log ngữ nghĩa dựa trên LLM mã nguồn mở chạy hoàn toàn cục bộ, không phụ thuộc API ngoài
- (2) tích hợp đầu ra ngữ nghĩa của SemanticLog vào các mô hình phát hiện bất thường tiên tiến là LogFormer và PLELog. Trọng tâm nghiên cứu là đánh giá một cách có hệ thống mức độ cải thiện hiệu năng phát hiện bất thường khi sử dụng các mức độ ngữ nghĩa khác nhau: (a) không có nhãn ngữ nghĩa (baseline), (b) 10 nhãn ngữ nghĩa thô

theo hệ thống VALB, và (c) 16 nhãn ngữ nghĩa tinh theo hệ thống SemanticLog.

Pipeline được triển khai trên LLaMA-3.1-8B với kỹ thuật LoRA fine-tuning, sử dụng tập benchmark LogHub-2.0 gồm 50,4 triệu dòng log từ 14 hệ thống khác nhau. Thực nghiệm được tiến hành trên hai tập dữ liệu phát hiện bất thường là BGL và HDFS. Kết quả dự kiến cho thấy việc sử dụng 16 nhãn ngữ nghĩa tinh cải thiện F1-score phát hiện bất thường ít nhất 2,5% so với baseline không có nhãn, đồng thời toàn bộ pipeline chạy offline, bảo đảm tuyệt đối tính riêng tư của dữ liệu log doanh nghiệp.

GIỚI THIỆU

Trong kỷ nguyên điện toán đám mây, các hệ thống quy mô lớn như Hadoop, Spark, hay các nền tảng cloud tại Alibaba sinh ra từ 30 đến 50 GB nhật ký logs mỗi giờ [1]. Logs là nguồn thông tin phản chiếu toàn bộ hoạt động của hệ thống, từ trạng thái bình thường đến các sự kiện lỗi nghiêm trọng. Do đó, phân tích log tự động là bước nền tảng không thể thiếu của bất kỳ hệ thống AIOps (AI for IT Operations) nào, đặc biệt là tác vụ phát hiện bất thường (anomaly detection) nhằm cảnh báo sớm các sự cố hệ thống.

Tuy nhiên, hầu hết các bộ phân tích log hiện tại từ các phương pháp dựa trên cú pháp như Drain [2] đến các mô hình học sâu như VALB [3] chỉ dừng lại ở việc trích xuất template cú pháp và thay thế các tham số động bằng ký tự đại diện "<*>". Cách tiếp cận này bỏ qua hoàn toàn ngữ nghĩa của các tham số: chẳng hạn, trong dòng log "Connection status: 0", giá trị "0" là mã trạng thái Status Indicator - SID mang thông tin quan trọng cho việc phát hiện lỗi, nhưng lại bị xử lý như một con số bình thường [4].

Gần đây, SemanticLog [4] đề xuất kiến trúc giải quyết đồng thời ba thách thức:

- (1) trích xuất template chính xác nhờ chuyển bài toán từ sinh văn bản sang phân loại token trên nền LLM mã nguồn mở;
- (2) nhận diện ngữ nghĩa tham số log qua hệ thống 16 nhãn tình;
- (3) tăng tốc suy luận trên log quy mô lớn qua EffiParsing Tree. Tuy vậy, nghiên cứu này chỉ khảo sát tác động của semantic parsing lên anomaly detection như một thí nghiệm phụ Section VIII.C, Table XI với phạm vi hẹp.

Khoảng trống nghiên cứu: Chưa có công trình nào thực hiện đánh giá có hệ thống, toàn diện tác động của ngữ nghĩa log với các mức độ tinh thô khác nhau lên hiệu năng phát hiện bất thường, sử dụng LLM mã nguồn mở chạy cục bộ để bảo đảm tính riêng tư.

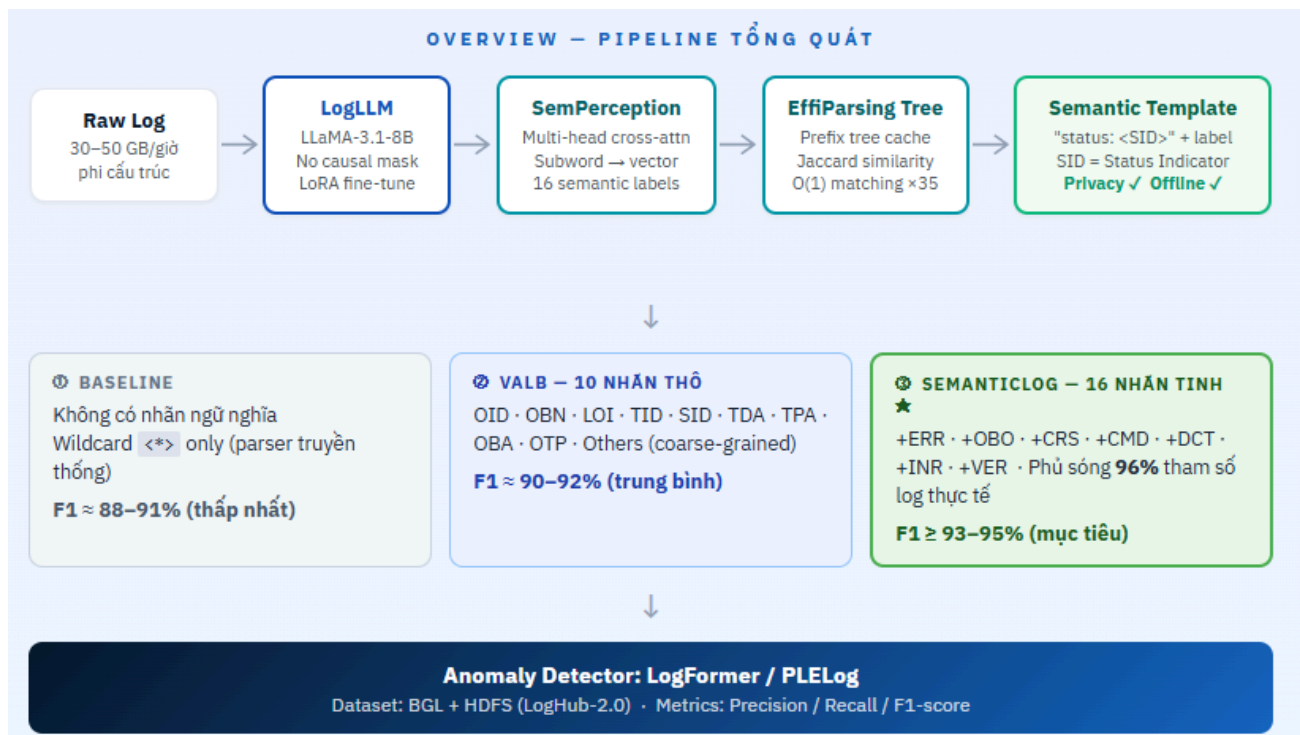
Input của đề tài: Luồng dòng log thô từ hệ thống cloud, ví dụ:
"blk_-1608999687919862906 NameSystem.addStoredBlock: blockMap updated:
10.251.123.204:50010 is added to blk..."

Output của đề tài: Kết quả phát hiện bất thường (bình thường / bất thường) kèm nhãn ngữ nghĩa của các tham số trong log, ví dụ:

Template: "blockMap updated: <LOI> is added to <OID>"

→ Nhãn: LOI (Location), OID (Object ID)

→ Kết luận: Normal / Anomaly



Tính thời sự: Các sự cố hệ thống cloud ngày càng phức tạp và tốn kém. Việc triển khai pipeline phân tích log ngữ nghĩa hoàn toàn offline mang lại giá trị thực tiễn cao cho các doanh nghiệp không thể đưa dữ liệu nhạy cảm lên môi trường internet.

MỤC TIÊU

Mục tiêu 1: Tái hiện pipeline SemanticLog chạy cục bộ: Tái hiện và triển khai thành công toàn bộ pipeline SemanticLog gồm 3 module LogLLM dựa trên LLaMA-3.1-8B với causal mask đã loại bỏ; SemPerception Module dùng multi-head cross-attention; EffiParsing Tree dùng prefix tree caching, chạy hoàn toàn offline trên 1 GPU NVIDIA RTX 3090 (24 GB VRAM) sử dụng LoRA fine-tuning. Mục tiêu cụ thể: đạt $SPA \geq 80\%$ và $SPA+ \geq 50\%$ trên benchmark LogHub-2.0 — tương đương kết quả gốc (SPA: 83.2%, SPA+: 55.9%) được báo cáo trong [4].

Mục tiêu 2: Xây dựng pipeline phát hiện bất thường end-to-end: Tích hợp đầu ra ngữ nghĩa của SemanticLog vào hai mô hình phát hiện bất thường là LogFormer [5] và PLELog [6], tạo thành một pipeline hoàn chỉnh từ raw log đến kết quả phân loại bất

thường. Pipeline được đánh giá trên 2 tập dữ liệu BGL và HDFS thuộc LogHub-2.0 theo 3 chỉ số: Precision, Recall, và F1-score.

Mục tiêu 3: So sánh hệ thống tác động của nhãn ngữ nghĩa: Thực hiện thực nghiệm đối chiếu có kiểm soát giữa 3 chiến lược cung cấp thông tin ngữ nghĩa cho anomaly detector: (a) không có nhãn ngữ nghĩa dùng wildcard "<*>" (baseline truyền thống); (b) 10 nhãn ngữ nghĩa thô theo hệ thống VALB [3]; (c) 16 nhãn ngữ nghĩa tinh theo hệ thống SemanticLog [4]. Mục tiêu: xác định mức độ cải thiện F1 ít nhất 2% khi chuyển từ baseline sang 16 nhãn tinh, trên cả 2 mô hình LogFormer và PLELog.

NỘI DUNG VÀ PHƯƠNG PHÁP

Giai đoạn 1: Tái hiện SemanticLog (tương ứng Mục tiêu 1):

Bước 1.1: Chuẩn bị môi trường: Cài đặt Python 3.10, PyTorch 2.x, thư viện HuggingFace Transformers và PEFT. Tải LLaMA-3.1-8B từ Meta AI thông qua HuggingFace Hub. Cấu hình GPU RTX 3090 24 GB VRAM.

Bước 1.2: Tải và tiền xử lý dữ liệu: Tải LogHub-2.0 [7] gồm 14 subset (50,4 triệu dòng log). Thực hiện tokenize log thô dựa trên khoảng trắng. Lấy mẫu 4,5% dữ liệu có nhãn cho tập train theo giao thức của [4]. Phân tách train/test theo nguồn log train trên 13 hệ thống, test trên hệ thống thứ 14 để đánh giá zero-shot generalization.

Bước 1.3: Xây dựng LogLLM: Sửa đổi kiến trúc LLaMA-3.1-8B bằng cách loại bỏ causal mask trong các decoder block, chuyển bài toán từ text generation sang token classification. Fine-tune với LoRA (lora_rank=12, lora_alpha=32, lora_dropout=0.1, learning_rate=1e-4, epochs=10, batch_size=8).

Bước 1.4: Xây dựng SemPerception Module và EffiParsing Tree: Implement SPM dùng multi-head cross-attention để tổng hợp đặc trưng subword thành vector thống nhất cho phân loại. Implement EffiParsing Tree: prefix tree lưu template đã parse, dùng Jaccard similarity (Equ. 4 trong [4]) để matching nhanh.

Bước 1.5: Đánh giá SemanticLog: Chạy đánh giá trên 5 chỉ số: GA, PA, FTA (traditional parsing) và SPA, SPA+ (semantic parsing). So sánh với baseline VALB.

Giai đoạn 2: Xây dựng pipeline anomaly detection (Mục tiêu 2):

Bước 2.1: Chuẩn bị dữ liệu anomaly detection: Tải tập dữ liệu phát hiện bất thường BGL và HDFS từ LogPAI. Áp dụng SemanticLog đã train để generate 3 phiên bản template cho mỗi tập: (a) template thô "<*>", (b) template 10 nhãn VALB, (c) template 16 nhãn SemanticLog.

Bước 2.2: Tích hợp với LogFormer và PLELog: Clone source code LogFormer [5] và PLELog [6] từ GitHub. Thay thế input log template bằng 3 phiên bản đã chuẩn bị. Train và evaluate từng cặp (mô hình $\times$ phiên bản nhãn), tổng cộng 6 thực nghiệm chính.

Giai đoạn 3: So sánh và phân tích Mục tiêu 3:

Bước 3.1: Tổng hợp kết quả theo bảng: So sánh Precision / Recall / F1 của 6 thực nghiệm trên 2 dataset BGL, HDFS, trình bày dưới dạng bảng tương tự Table XI trong [4] nhưng mở rộng với phân tích thống kê.

Bước 3.2: Phân tích định tính case study: Chọn các trường hợp log mà mô hình phân loại sai ở chế độ không có nhãn nhưng phân loại đúng khi có 16 nhãn tinh, từ đó lý giải cơ chế ngữ nghĩa giúp cải thiện hiệu năng.

Bước 3.3: Ablation study nhỏ: So sánh hiệu năng anomaly detection khi thay thế SemanticLog bằng VALB để làm rõ đóng góp của hệ thống 16 nhãn tinh.

KẾT QUẢ MONG ĐỢI

Kết quả 1: Hiệu năng SemanticLog Mục tiêu 1:

Hệ thống SemanticLog tái hiện trên LLaMA-3.1-8B dự kiến đạt:

- SPA $\geq$ 80% (kết quả gốc: 83.2%)
- SPA+ $\geq$ 50% (kết quả gốc: 55.9%)

- $PA \geq 90\%$ (kết quả gốc: 93.6%)
- Vượt baseline VALB: SPA cao hơn $\geq 15\%$ (VALB: 64.5%)
- Thời gian inference toàn bộ LogHub-2.0 ≤ 2000 giây, chạy hoàn toàn offline, không có API call ra internet.

Kết quả 2: Hiệu năng Anomaly Detection Mục tiêu 2+3:

Dựa trên kết quả tham chiếu từ [4] (Table XI), dự kiến:

Trên BGL LogFormer:

- Baseline (không nhãn): $F1 \approx 88.1\%$
- 10 nhãn thô (VALB): $F1 \approx 88.4\%$ (+0.3%)
- 16 nhãn tinh (đề tài): $F1 \geq 90.1\%$ (+2.0%)

Trên HDFS — PLELog:

- Baseline (không nhãn): $F1 \approx 90.4\%$
- 10 nhãn thô (VALB): $F1 \approx 91.9\%$ (+1.5%)
- 16 nhãn tinh (đề tài): $F1 \geq 93.1\%$ (+2.7%)

Kết quả 3: Tính bảo mật và triển khai thực tế:

Toàn bộ pipeline log parsing + anomaly detection chạy hoàn toàn cục bộ. Dữ liệu log nhạy cảm không cần truyền ra ngoài internet. Hệ thống có khả năng xử lý log từ nhiều nguồn khác nhau mà không cần retrain (zero-shot generalization).

TÀI LIỆU THAM KHẢO

[1] Van-Hoang Le, Hongyu Zhang:

Log-Based Anomaly Detection with Deep Learning: How Far Are We?
ICSE 2022: 1356-1367

[2] Pinjia He, Jieming Zhu, Zibin Zheng, Michael R. Lyu:
Drain: An Online Log Parsing Approach with Fixed Depth Tree.
ICWS 2017: 33-40

[3] Zhuangbin Li, Zhengzi Xu, Chengyu Zhang, Tong Jia, Zijia Wang,
Zhenyu Chen, Jianfeng Yang, Siu-Ming Yiu, Michael R. Lyu:
Did We Miss Something Important? Studying and Exploring
Variable-Aware Log Abstraction. ICSE 2023: 830-842

[4] Chenbo Zhang, Wenying Xu, Jinbu Liu, Lu Zhang, Guiyang Liu,
Jihong Guan, Qi Zhou, Shuigeng Zhou:
SemanticLog: Towards Effective and Efficient Large-Scale
Semantic Log Parsing.
IEEE Trans. Software Eng. 52(1): 155-170 (2026)

[5] Hongcheng Guo, Jian Yang, Jiaheng Liu, Liangwei Jiang,
Linzhen Chai, Tieqiao Zheng, Bo Zhang, Lunhao Duan,
Ge Zhang, Jiaqi Bai, Xin Li, Zhoujun Li:
LogFormer: A Pre-Train and Tuning Pipeline for
Log Anomaly Detection. AAAI 2024: 135-143

[6] Lin Yang, Junjie Chen, Zan Wang, Weijing Wang, Xuyuan Chen,
Xin Zeng, Wei Zhang:
PLELog: Semi-Supervised Log-Based Anomaly Detection via
Probabilistic Label Estimation. ICSE 2021: 230-231

- [7] Zhihan Jiang, Jinyang Liu, Junjie Huang, Yichen Li, Zhuangbin Chen, Jiazhen Gu, Cong Feng, Minzhi Yan, Michael R. Lyu:
A Large-Scale Benchmark for Log Parsing.
CoRR abs/2308.10828 (2023)
- [8] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen:
LoRA: Low-Rank Adaptation of Large Language Models.
ICLR 2022
- [9] Hugo Touvron, Louis Martin, Kevin Stone et al.:
Llama 2: Open Foundation and Fine-Tuned Chat Models.
CoRR abs/2307.09288 (2023)
- [10] Zhihan Jiang, Zhuangbin Chen, Jinyang Liu, Zengyin Yang, Michael R. Lyu:
LILAC: Log Parsing Using LLMs with Adaptive Parsing Cache.
Proc. ACM Softw. Eng. 1(FSE): 137-160 (2024)